Condicionales, estructuras repetitivas y funciones

Objetivo: Conocer la ruta de la clase y lo que podrás resolver al terminarla.

Parte 1 · Teoría

decisiones, ciclos, iteradores de arrays y funciones

Parte 2 · Práctica

checkpoints → ejercicios guiados → para la casa → reto integrador

Punto de partida

Partimos de lo que ya sabes: estructura básica de HTML, variables, tipos de datos y operadores

Meta

que tu programa decida, repita y se organice en piezas reutilizables



Parte 1 · Teoría: if-else y if else-if

Objetivo: Elegir qué código se ejecuta según una condición.

if-else

ejecuta un bloque si la condición es verdadera y otro si es falsa

```
const hora = 14;
if (hora < 12) {
  console.log("Buenos días");
} else {
  console.log("Buenas tardes");
}
```

Salida: Buenas tardes

if else-if else

encadena varias condiciones; se ejecuta solo el primer bloque verdadero

```
const calificacion = 75;
if (calificacion >= 90) {
  console.log("A");
} else if (calificacion >= 80) {
  console.log("B");
} else if (calificacion >= 70) {
  console.log("C");
} else {
  console.log("Reprobado");
}
```

Salida: C

Operador ternario y switch

Objetivo: Escribir decisiones cortas y comparar un valor contra varios casos.

Ternario

condición ? valorSiTrue : valorSiFalse, un if-else de una sola línea que produce un valor

```
const numero = 7;
const mensaje = numero % 2 === 0
  ? "Es par"
  : "Es impar";
console.log(mensaje);
```

Salida: Es impar

switch

evalúa la expresión una vez y la compara con cada case usando igualdad estricta

break cierra el caso; default se ejecuta si ninguno coincide

```
const dia = "Lunes";
switch (dia) {
  case "Lunes":
    console.log("Comienzo de semana");
    break;
  default:
    console.log("Día intermedio");
}
```

Salida: Comienzo de semana

🤔 Pregunta rápida: decisiones

Objetivo: Comprobar cómo se evalúan las condiciones.

1 ¿Qué imprime este código?

```
const nota = 85;
if (nota >= 70) {
  console.log("C");
} else if (nota >= 80) {
  console.log("B");
}
```

2 ¿Qué imprime este código?

```
const opcion = "1";
switch (opcion) {
  case 1:
    console.log("Uno");
    break;
  default:
    console.log("Otro");
}
```

3 ¿Qué imprime este código?

```
console.log("5" == 5, "5" === 5);
```

Ciclos for y for...of

Objetivo: Repetir un bloque y recorrer los valores de una colección.

for

repite un número específico de veces (inicio; condición; incremento)

```
for (let i = 1; i <= 3; i++) {
  console.log(i);
}
```

Salida: 1 2 3

for...of

recorre los valores de un iterable, como un array o un string

```
const colores = ["rojo", "azul"];
for (const color of colores) {
  console.log(color);
}
```

Salida: rojo azul

Ciclos for...in y while

Objetivo: Recorrer las propiedades de un objeto y repetir mientras se cumpla una condición.

for...in

recorre los nombres de las propiedades de un objeto; también puede recorrer propiedades heredadas del prototipo

```
const persona = {
  nombre: "Juan",
  ciudad: "Bucaramanga"
};
for (const propiedad in persona) {
  console.log(`${propiedad}: ${persona[propiedad]}`);
}
```

Salida: nombre: Juan ciudad: Bucaramanga

while

revisa la condición antes de cada vuelta; dentro del bloque algo debe acercarla a ser falsa

```
let i = 1;
while (i <= 3) {
  console.log(`Cuadrado de ${i}: ${i * i}`);
  i++;
}
```

Salida: Cuadrado de 1: 1 Cuadrado de 2: 4 Cuadrado de 3: 9

do...while, break y continue

Objetivo: Ejecutar al menos una vez y controlar un ciclo desde adentro.

do...while

ejecuta el bloque primero y revisa la condición después

```
let intentos = 0;
do {
  intentos++;
  console.log(`Intento ${intentos}`);
} while (intentos < 0);
```

Salida: Intento 1

continue y break

continue: salta a la siguiente vuelta

break: sale del ciclo o del switch

```
for (let i = 1; i <= 5; i++) {
  if (i === 2) { continue; }
  if (i === 4) { break; }
  console.log(i);
}
```

Salida: 1 3

🤔 Pregunta rápida: ciclos

Objetivo: Predecir cuántas vueltas da un ciclo y qué recorre.

1 ¿Qué imprime este código?

```
let n = 10;
do {
  console.log(n);
  n++;
} while (n < 5);
```

2 ¿Qué imprime este código?

```
const letras = ["a", "b"];
for (const x in letras) {
  console.log(x);
}
```

3 ¿Qué imprime este código? ¿Y si cambias break por continue?

```
for (let i = 0; i < 3; i++) {
 if (i === 1) { break; }
 console.log(i);
}
```

Iteradores: forEach y map

Objetivo: Actuar sobre cada elemento y transformar un array en otro nuevo.

Los iteradores son métodos de array que reciben una función y la aplican a los elementos

forEach

ejecuta una acción por elemento; no crea un array nuevo

```
const frutas = ["Manzana", "Uva"];
frutas.forEach(function (fruta) {
  console.log(fruta);
});
```

Salida: Manzana Uva

map

crea un array nuevo con cada elemento transformado; el original no cambia

```
const palabras = ["Hola", "Mundo"];
const mayusculas = palabras.map(function (p) {
  return p.toUpperCase();
});
console.log(mayusculas);
console.log(palabras);
```

Salida: ['HOLA', 'MUNDO'] ['Hola', 'Mundo']

Iteradores: filter y every

Objetivo: Quedarse con algunos elementos y verificar si todos cumplen una condición.

filter

crea un array nuevo con los elementos que cumplen la condición

```
const numeros = [5, 12, 8, 20];
const mayores = numeros.filter(function (n) {
  return n > 10;
});
console.log(mayores);
```

Salida: [12, 20]

every

devuelve true si todos cumplen y false si al menos uno no cumple

```
const palabras = ["casa", "perro", "sol"];
const largas = palabras.every(function (p) {
  return p.length > 3;
});
console.log(largas);
```

Salida: false

Iteradores: indexOf y reduce

Objetivo: Ubicar un elemento y combinar todo el array en un único valor.

indexOf

devuelve la primera posición de un elemento, o -1 si no está

```
const frutas = ["manzana", "pera", "uva"];
console.log(frutas.indexOf("pera"));
console.log(frutas.indexOf("kiwi"));
```

Salida: 1 -1

reduce

acumula un resultado, como una suma o un promedio; el último argumento es el valor inicial

```
const numeros = [1, 2, 3, 4];
const suma = numeros.reduce(function (acumulador, numero) {
  return acumulador + numero;
}, 0);
console.log(suma);
```

Salida: 10

`reduce` recorre el array elemento por elemento y va guardando un resultado parcial hasta dejar **un solo valor**. En este caso, suma todos los números y obtiene 10.

Las piezas del código

- `acumulador`: guarda el resultado que se lleva hasta el momento.
- `numero`: es el elemento del array que toca en esa vuelta.
- `return acumulador + numero`: lo que devuelve la función se convierte en el nuevo valor de `acumulador` en la siguiente vuelta.
- `0`: es el valor inicial, es decir, con lo que empieza `acumulador`. Va después de la función, y por eso aparece como `}, 0);`.

🤔 Pregunta rápida: iteradores

Objetivo: Distinguir qué devuelve cada iterador.

1 ¿Qué imprime este código?

```
const nums = [1, 2, 3];
const r = nums.forEach(function
(n) {
  return n * 2;
});
console.log(r);
```

2 ¿Qué imprime este código?

```
const p = ["sol", "mar"];
const m = p.map(function (w) {
  return w.toUpperCase();
});
console.log(p[0], m[0]);
```

3 ¿Qué imprime este código?

```
console.log([5, "5"].indexOf("5"), [1,
2].indexOf(3));
```

Funciones declarativas

Objetivo: Definir, invocar y reutilizar una función.

¿Qué es una función?

Una función es un bloque que realiza una tarea cuando se llama; ayuda a organizar y reutilizar el código

Partes

function, nombre, parámetros, bloque de código e invocación con argumentos

return

detiene la función y devuelve un valor al lugar donde fue llamada

Sin return

la función devuelve undefined

```
function miFuncion(p1, p2) {
  return p1 * p2;
}
console.log(miFuncion(2, 2));
console.log(miFuncion(5, 3));
```

Salida: 4 15

Hoisting, argumentos y operador ()

Objetivo: Anticipar qué pasa según cómo se llama una función.

Hoisting

una función declarativa se puede llamar antes de su declaración. Funciona porque JavaScript no ejecuta el código directamente línea por línea: **primero prepara el código y después lo ejecuta**

```
console.log(doble(4));
function doble(n) {
  return n * 2;
}
```

Salida: 8

Argumentos y operador ()

Los argumentos incorrectos dan resultados erróneos: + con un texto concatena y un argumento faltante llega como undefined

Number() convierte un texto a número

Sin () obtienes la definición de la función, no su ejecución

```
function sumar(a, b) { return a + b; }
console.log(sumar("5", 1));
console.log(sumar(Number("5"), 1));
console.log(sumar(5));
console.log(sumar);
```

Salida: 51 6 NaN function sumar(a, b) { return a + b; }

El `NaN` sale de la línea `sumar(5)`, porque la función espera dos argumentos y solo recibe uno.

1. La función calcula `5 + undefined`.
2. Como ninguno de los dos es texto, `+` intenta sumar números y convierte `undefined` a número.
3. `undefined` no tiene equivalente numérico, así que se convierte en `NaN`.

Expresiones, anónimas y callbacks

Objetivo: Guardar funciones en constantes y pasarlas como argumento.

Función de expresión y anónima

Función de expresión: se asigna a una variable o constante y debe definirse antes de invocarla

Función anónima: no tiene nombre; se usa en una expresión o como callback

Este ejemplo es las dos cosas a la vez: es de expresión porque se asigna a `triple`, y es anónima porque después de `function` no hay nombre. `triple` es el nombre de la constante que la guarda, no un nombre propio de la función.

```
const triple = function (n) {
  return n * 3;
};
console.log(triple(2));
```

Salida: 6

Callback

función que se pasa como argumento a otra función, que la ejecuta después

```
function cuadrado(numero) {
  return numero * numero;
}
const numeros = [1, 2, 3];
console.log(numeros.map(cuadrado));
```

Salida: [1, 4, 9]

Función	Quién la creó	Qué papel cumple
cuadrado	Tú, en el código	Es el **callback**: calcula el cuadrado de un número
`map`	JavaScript (método de los arrays)	Recibe `cuadrado` y la ejecuta con cada elemento

🤔 Pregunta rápida: funciones

Objetivo: Razonar sobre orden de ejecución y valores devueltos.

1 ¿Qué imprime este código?

```
saludar();
function saludar() {
  console.log("Hola");
}
```

2 ¿Qué pasa al ejecutar este código?

```
despedir();
const despedir = function () {
  console.log("Chao");
};
```

3 ¿Qué imprime este código?

```
function total(precio, cantidad) {
 return precio * cantidad;
 console.log("Listo");
}
console.log(total(2));
```

Funciones flecha

Objetivo: Escribir funciones cortas con sintaxis concisa.

Estructura

(argumentos) => expresión

Un solo argumento

Con un solo argumento, los paréntesis son opcionales

Sin llaves

Si el cuerpo es una sola expresión sin llaves, el return es implícito

Con llaves

Si usas llaves, debes escribir return

Uso ideal

Son ideales para callbacks y funciones de un solo uso

```
const sumaUno = a => a + 1;
const resta = (a, b) => a - b;
const cuadrado = (numero) => { return numero * numero; };
const esMayorQueDiez = (numero) => numero > 10;
//Aca se invocan las funciones
console.log(sumaUno(1), resta(5, 3), cuadrado(5), esMayorQueDiez(20));
```

Salida: 2 2 25 true

IIFE y ámbito

Objetivo: Encapsular variables y reconocer dónde existe cada una.

IIFE

función que se ejecuta tan pronto se define; los paréntesis finales reciben los argumentos

Encapsula variables en un ámbito local y evita contaminar el ámbito global. Da `undefined` porque `saludo` solo existe dentro de la IIFE

```
(function (nombre) {
  const saludo = `¡Hola, ${nombre}!`;
  console.log(saludo);
})("Pepito");
console.log(typeof saludo);
```

Salida: ¡Hola, Pepito! undefined

Ámbito de bloque y clausura

const y let solo existen dentro del bloque donde se declaran

```
if (true) {
  const mensaje = "dentro";
  console.log(mensaje);
}
console.log(mensaje);
```

Salida: dentro Uncaught ReferenceError: mensaje is not defined

Clausura: cuando una función devuelve otra, la interna conserva acceso a su ámbito original

🤔 Pregunta rápida: flecha y ámbito

Objetivo: Detectar errores comunes de retorno y alcance.

1 ¿Qué imprime este código?

```
const doble = (n) => { n * 2 };
console.log(doble(4));
```

2 ¿Qué pasa al ejecutar este código?

```
(function () {
  const clave = "abc";
})();
console.log(clave);
```

3 ¿Qué pasa al ejecutar este código?

```
for (let i = 0; i < 2; i++) { }
console.log(i);
```

Chuleta: ¿qué necesito y qué uso?

Objetivo: Conectar cada problema típico con su herramienta.

Necesito...	Uso...
Elegir entre caminos o rangos	if-else, if else-if else o ternario
Comparar un valor con casos fijos	switch
Repetir un número conocido de veces	for
Repetir hasta que algo cambie	while o do...while
Recorrer valores o propiedades	for...of o for...in
Saltar una vuelta o salir del ciclo	continue o break
Hacer algo con cada elemento	forEach
Transformar todos los elementos	map
Quedarme con algunos elementos	filter
Saber si todos cumplen	every
Encontrar una posición	indexOf
Obtener un único valor	reduce
Reutilizar una tarea	función declarativa, de expresión o flecha
Aislar variables	IIFE

Parte 2 · Práctica: orden de trabajo y reglas de juego

Objetivo: Conocer cómo vamos a trabajar y qué se revisa en cada ejercicio.

Orden: ✅ Checkpoints → 🏫 Guiados → 🏠 Para la casa → 🔥 Reto

Reglas:

- Usa index.html + js/main.js, con el script vinculado justo antes de </body>
- No modifiques los datos ni el código base, salvo donde el ejercicio lo indique
- Comenta cada requisito con // R1
- Usa const por defecto y let solo si hay que reasignar; no uses var
- Usa === y !== en las comparaciones
- No uses librerías externas
- Verifica cada resultado en la consola



✅ Checkpoint práctico 1: Domicilio

Objetivo: Combinar conversión de tipos, ternario y if else-if en un pedido real.

Retos

1. Ejecuta el código y explica por qué no imprime 50000.
2. Guarda en subtotal el valor de subtotalTexto convertido a número.
3. Con un ternario, deja el domicilio gratis desde 50000 e imprime el total.
4. Con if else-if else, imprime "Pedido grande" desde 100000, "Pedido mediano" desde 50000 y "Pedido pequeño" en otro caso.

Código base

```
const subtotalTexto = "45000";
const domicilio = 5000;
console.log(subtotalTexto + domicilio);
```

✅ Checkpoint práctico 2: Turnos de la clínica

Objetivo: Combinar for...of, continue, break y un contador.

Retos

1. Recorre turnos con for...of e imprime "Atendiendo" y el turno.
2. Salta los turnos "cancelado" con continue.
3. Detén el recorrido con break al llegar a "CIERRE".
4. Cuenta los turnos atendidos e imprime "Atendidos: n".
5. Explica por qué nunca aparece D4 y por qué el contador no puede ser const.

Código base

```
const turnos = [
 "A1",
 "B2",
 "cancelado",
 "C3",
 "CIERRE",
 "D4"
];
```

✅ Checkpoint práctico 3: Promoción de la tienda

Objetivo: Combinar funciones flecha, map, filter y reduce.

Retos

1. Ejecuta el código y explica por qué aparece undefined.
2. Corrige solo conDescuento para obtener la mitad de cada precio.
3. Con filter, guarda los nuevos precios desde 5000.
4. Con reduce, suma esos precios.
5. Imprime precios y explica por qué sigue igual.

Código base

```
const precios = [12000, 8000, 30000, 5000];
const conDescuento = (precio) => { precio / 2 };
const nuevos = precios.map(conDescuento);
console.log(nuevos);
```

🏫 Ejercicio guiado 1: Factura de la tienda

Objetivo: Construir una factura con funciones e iteradores sin alterar los datos.

Requisitos

- R1: Función declarativa subtotal(item); si falta la cantidad, asume 1.
- R2: Con map, crea subtotales.
- R3: Con forEach, imprime cada producto y su subtotal.
- R4: Con reduce, calcula e imprime el total.
- R5: Con un ternario, deja el envío en 0 desde 60000; si no, usa ENVIO.

Salida esperada

Café: $36000
Panela: $13500
Arepas: $6000
Queso: $12000
Total: $67500
Envío: $0
A pagar: $67500

Código base

```
const carrito = [
 { producto: "Café", precio: 18000, cantidad: 2 },
 { producto: "Panela", precio: 4500, cantidad: 3 },
 { producto: "Arepas", precio: "6000", cantidad: 1 },
 { producto: "Queso", precio: 12000 },
];
const ENVIO = 5000;
```

🏫 Ejercicio guiado 2: Notas del curso

Objetivo: Analizar las notas de un grupo combinando funciones, ciclos e iteradores.

Requisitos

- R1: Flecha estado(nota) con ternario; desde NOTA_MINIMA se aprueba.
- R2: Con for...of, imprime nombre, nota y estado.
- R3: Con filter, imprime cuántos aprobaron.
- R4: Función de expresión promedio(lista) con reduce.
- R5: Con every, indica si todos aprobaron.
- R6: Crea nombres con map y busca "Sara" y "sara" con indexOf.

Salida esperada

```
Ana (4.5): Aprobó
Luis (2.9): Reprobó
Sara (3): Aprobó
Juan (3.8): Aprobó
Aprobados: 3
Promedio: 3.55
¿Todos aprobaron? false
2 -1
```

Código base

```
const estudiantes = [
 { nombre: "Ana", nota: 4.5 },
 { nombre: "Luis", nota: 2.9 },
 { nombre: "Sara", nota: 3 },
 { nombre: "Juan", nota: 3.8 },
];
const NOTA_MINIMA = 3;
```

🏠 Para la casa

Objetivo: Reforzar lo visto con tres casos reales de dificultad similar a los guiados.

Caso	Temas que combina
Parqueadero	switch, funciones declarativa y flecha, Number(), for...of, ternario, map, reduce
Inventario de la papelería	filter, forEach, every, map, indexOf, reduce, Number()
Gastos del mes	for...in, switch, función declarativa, Number(), ternario, while

El enunciado completo de cada caso va en el README.

🔥 Reto integrador: Cierre de caja de la cafetería

Objetivo: Combinar la mayoría de los temas en un programa completo con forma de evaluación.

Requisito	Qué debe hacer	Concepto que evalúa
R1	Encierra el programa en una IIFE; fuera, imprime typeof totalDia	IIFE, ámbito
R2	Crea valorVenta(venta), declarada al final y usada antes; si falta la cantidad, asume 1	Declarativa, hoisting, ternario, conversión
R3	Deja solo las ventas no anuladas	filter
R4	Calcula el valor de cada venta y el total del día	map, reduce
R5	Imprime el detalle y el total	for...of
R6	Acumula por método de pago e imprime el resumen; otro método va en "otro"	switch, objeto, for...in
R7	Imprime la venta con la que se alcanza META	while
R8	Verifica que todas tengan valor, ubica "bono" e imprime si se cumplió la meta	every, indexOf, ternario

🔥 Reto integrador: código base, entregable y rúbrica

Objetivo: Saber qué datos usar, qué entregar y cómo se califica.

Entregable y rúbrica

Entregable: carpeta comprimida con index.html y js/main.js, con cada requisito comentado (// R1).

Criterio	Peso
Decisiones: R2 y R6	20 %
Ciclos: R5, R6 y R7	20 %
Iteradores: R3, R4 y R8	25 %
Funciones y ámbito: R1 y R2	20 %
Reglas de juego y salida correcta	15 %

Código base

```
const ventas = [
 { id: 1, producto: "Empanada", precio: 2500,
 cantidad: 4, metodo: "efectivo", anulada: false },
 { id: 2, producto: "Jugo", precio: "3500",
 cantidad: 2, metodo: "tarjeta", anulada: false },
 { id: 3, producto: "Almuerzo", precio: 12000,
 cantidad: 1, metodo: "efectivo", anulada: true },
 { id: 4, producto: "Café", precio: 2000,
 cantidad: 3, metodo: "transferencia", anulada: false },
 { id: 5, producto: "Almuerzo", precio: 12000,
 cantidad: 2, metodo: "tarjeta", anulada: false },
 { id: 6, producto: "Gaseosa", precio: 3000,
 metodo: "bono", anulada: false },
];
const META = 40000;
```



Resumen y lista de chequeo

Objetivo: Recoger las ideas clave y revisar el trabajo antes de entregarlo.

Resumen

- Decidir: if, ternario y switch
- Repetir: for, for...of, for...in, while y do...while
- Transformar datos: forEach, map, filter, every, indexOf y reduce
- Organizar: funciones declarativas, de expresión, flecha e IIFE

Antes de entregar

- El script está vinculado justo antes de </body>
- Cada requisito tiene su comentario // R1
- Usé const por defecto, let solo al reasignar y ningún var
- Comparé con === y !==
- No modifiqué los datos ni el código base
- No usé librerías externas
- Revisé textos, undefined y NaN
- La salida en la consola coincide con la esperada